SIMATS ENGINEERING

MCQ Test

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*
Assessment Tool Weightage: 20%

Sl. No	MCQ Questions	Bloom's Level
1	A team applies systematic test design techniques during development. This reflects: a) Debugging b) Testing as an engineering activity c) Maintenance d) Deployment	3
2	If a project skips defined testing processes, what is the likely outcome? a) Faster delivery with high quality b) Increased defects in production c) Reduced testing cost only d) Improved documentation	3
3	Selecting test cases based on requirements demonstrates: a) Random testing b) Process-driven testing c) Ad hoc testing d) Maintenance testing	3
4	Applying equivalence partitioning is an example of: a) Debugging b) Test design technique c) Coding standard d) Deployment activity	3
5	A tester identifies missing requirements during testing. This shows: a) Coding error b) Process gap c) User error d) Hardware issue	3
6	Following a structured testing lifecycle ensures: a) Random defect detection b) Improved software quality c) Less documentation d) No defects	3
7	Which activity best represents testing as a process? a) Writing code b) Executing test cases systematically c) Deploying software d) User training	3
8	A tester reuses test cases from a repository. This supports: a) Efficiency in testing b) Increased coding errors c) Reduced coverage d) Random testing	3
9	If defects are tracked and analyzed, it improves: a) Coding speed b) Software quality process c) Deployment time d) User interface	3
10	Applying boundary value analysis helps to: a) Increase documentation b) Detect edge-case defects c) Reduce execution time d) Avoid testing	3
11	A tester validates system behavior against requirements. This is: a) Verification b) Validation c) Debugging d) Coding	3

12	Detecting defects before release reduces: a) Development effort b) Maintenance cost c) Coding standards d) Testing scope	3
13	Applying testing principles ensures: a) Zero defects guarantee b) Early defect detection c) No testing needed d) Faster coding	3
14	Which principle suggests testing all inputs is impossible? a) Exhaustive testing b) Exhaustive testing is impossible c) Early testing d) Defect clustering	3
15	A tester focuses on frequently failing modules. This uses: a) Random testing b) Defect clustering principle c) Regression testing d) Smoke testing	3
16	If testing starts early in SDLC, it helps: a) Reduce defects cost b) Increase coding errors c) Delay project d) Skip documentation	3
17	A tester designs test cases independently from developers. This ensures: a) Bias b) Objectivity c) Delay d) Redundancy	3
18	A tester reviews requirements for errors. Role performed is: a) Developer b) Tester c) Manager d) User	3
19	Reporting defects clearly is part of: a) Coding b) Tester's responsibility c) Deployment d) Maintenance	3
20	A tester collaborates with developers to fix bugs. This shows: a) Isolation b) Teamwork c) Delay d) Redundancy	3
21	A requirement ambiguity causes defects. Origin is: a) Coding b) Requirements phase c) Testing phase d) Deployment	3
22	Incorrect algorithm implementation leads to: a) Requirement defect b) Design defect c) Coding defect d) Testing defect	3
23	Missing validation checks are example of: a) Design defects b) Coding defects c) Requirement defects d) Deployment defects	3
24	Poor UI design causing confusion is: a) Coding defect b) Design defect c) Requirement defect d) Hardware defect	3
25	A system crash due to memory leak is: a) Requirement defect b) Coding defect c) Design defect d) Documentation defect	3
26	Classifying defects based on severity helps: a) Ignore defects b) Prioritize fixing c) Delay release d) Increase cost	3
27	A critical defect impacts: a) Minor feature b) Core functionality c) UI color d) Documentation	3
28	A minor defect example is: a) System crash b) Login failure c) Typo error d) Data loss	3
29	Defect classification helps testers to: a) Ignore bugs b) Organize defects c) Skip testing d) Increase errors	3

30	A defect repository is used to: a) Store code b) Store test cases c) Store defect details d) Store UI designs	3
31	Reusing defect data helps in: a) Predicting issues b) Increasing bugs c) Reducing tests d) Skipping phases	3
32	A defect repository improves: a) Code complexity b) Testing efficiency c) User interface d) Deployment speed	3
33	Logging defects with details ensures: a) Confusion b) Traceability c) Delay d) Redundancy	3
34	A tester uses past defect data to design tests. This is: a) Random testing b) Experience-based testing c) Debugging d) Deployment	3
35	Incorrect login validation is an example of: a) Functional defect b) Performance defect c) Security defect d) UI defect	3
36	Slow response time is a: a) Functional defect b) Performance defect c) UI defect d) Coding standard	3
37	Unauthorized data access indicates: a) UI defect b) Security defect c) Functional defect d) Design defect	3
38	Missing button functionality is: a) Functional defect b) Security defect c) Performance defect d) Hardware defect	3
39	Poor alignment in UI is: a) Functional defect b) UI defect c) Security defect d) Design defect	3
40	Applying testing principles avoids: a) Coding b) Redundant tests c) Deployment d) Maintenance	3
41	A tester validates outputs using expected results. This is: a) Debugging b) Test execution c) Coding d) Maintenance	3
42	If defects are not tracked, it leads to: a) Better quality b) Poor traceability c) Faster delivery d) No impact	3
43	Applying test process ensures: a) Random results b) Structured testing c) No defects d) No planning	3
44	Test case design based on requirements ensures: a) Low coverage b) High coverage c) No defects d) Less effort	3
45	Early testing reduces: a) Defect cost b) Code size c) UI design d) Deployment time	3
46	A tester identifies repeated defects. This indicates: a) Good design b) Poor process c) No issue d) Perfect testing	3
47	Reviewing test results helps: a) Ignore defects b) Improve quality c) Skip testing d) Reduce effort	3
48	A tester applies regression testing to: a) Add features b) Ensure fixes work c) Deploy software d) Design UI	3
49	Identifying root cause of defects helps: a) Increase bugs b) Prevent recurrence c) Delay testing d) Skip coding	3
50	A test plan ensures: a) Random execution b) Organized testing c) No testing d) No defects	3
51	Applying defect classification improves: a) Testing cost b) Fix prioritization c) Coding errors d) UI design	3

52	A tester uses historical defect trends. This helps in: a) Ignoring defects b) Risk-based testing c) Deployment d) Coding	3
53	Testing ensures product meets: a) Developer needs b) User requirements c) Tester goals d) Manager plans	3
54	A defect due to wrong requirement interpretation is: a) Coding defect b) Requirement defect c) UI defect d) Hardware defect	3
55	Applying process standards ensures: a) Poor quality b) Consistent quality c) No testing d) Random output	3
56	A tester documents defects clearly. This helps: a) Developers fix quickly b) Increase errors c) Delay release d) Reduce quality	3
57	A defect repository is analyzed to: a) Increase cost b) Improve testing strategy c) Skip testing d) Reduce quality	3
58	A tester applies black-box testing. This focuses on: a) Internal code b) External behavior c) Hardware d) Deployment	3
59	Defects found in testing phase indicate: a) Perfect design b) Earlier phase issues c) No issues d) Hardware problems	3
60	Applying structured testing improves: a) Random results b) Product reliability c) Coding errors d) Deployment delay	3

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	30%	Demonstrates thorough understanding; answers all questions accurately	Shows good understanding with few errors	Partial understanding; several incorrect responses	Lacks understanding; majority of answers incorrect
Accuracy of Answers	25%	All answers are correct	Most answers are correct with minor mistakes	Some correct answers; frequent errors	Mostly incorrect answers
Application of Knowledge	20%	Correctly applies concepts to problem-based questions	Applies concepts with minor errors	Limited ability to apply concepts	Unable to apply concepts
Time Management	15%	Completes quiz well within time efficiently	Completes on time with minor delays	Requires extra time or rushes through questions	Unable to complete within given time
Question Interpretation	10%	Interprets all questions correctly and responds appropriately	Misinterprets a few questions	Often misinterprets questions	Unable to understand questions

NAME: PAVITHRA.IC

REG NO: 192411027

COURSE NAME: Software Testing

Col- AT3

1. a) predicting issues
2. b) product reliability
3. b) improve testing strategy
4. b) Redundant tests
5. b) Experience-based testing
6. c) store defect details
7. b) Requirement defect
8. b) organized testing
9. b) core functionality
10. b) Testing as an engineering activity
11. b) Design defect
12. b) UI defect
13. b) Teamwork
14. b) Executing test cases systematically
15. b) process-driven testing
16. b) High coverage
17. b) validation
18. b) Improve quality
19. b) organize defects?
20. b) Testers' responsibility
21. a) External behavior

60/60

22. a) Functional de
23. b) Earlier phase issues
24. b) Traceability
25. c) Typo error
26. b) Testing efficiency
27. a) Efficiency in testing
28. b) process gap
29. b) Test execution
30. b) Test design technique
31. b) User requirements
32. b) performance defect
33. b) prevent recurrence
34. b) Ensure fixes work
35. b) objectivity
36. b) increased defects in production
37. b) structured testing
38. b) poor traceability
39. a) Defect cost
40. b) Early defect detection
41. b) Requirements phase
42. b) prioritize fixing
43. b) Exhaustive testing is impossible
44. b) consistent quality
45. a) Reduce defects cost
46. b) Software quality process
47. b) Maintenance cost
48. b) Defect clustering principle
49. b) Risk-based testing
50. b) security defect
51. c) coding defect
52. b) improved software quality
53. a) Functional defect
54. b) Detect edge-case defects
55. b) Fix prioritization
56. a) Developers fix quickly
57. b) Tester
58. b) Poor process
59. b) coding defect
60. b) coding defect